

HikeAlong

Project Documentation

Subject: Software Design

Student: Butuza Raisa Briana

1. Requirements Analysis

1.1 Assignment Specification

The objective of this project is to develop a full-stack application for managing an **event reservation system**.

The application allows:

- Users to browse events and reserve seats
- Organizers to create and manage events

Each user has a defined role (**USER** or **ORGANIZER**) which determines the actions they are allowed to perform.

The system is implemented using:

- **Spring Boot** for the backend
- **Angular** for the frontend

The application ensures:

- Data integrity
- Input validation
- Business rule enforcement
- Clear error handling

1.2 Functional Requirements

The application has the following functional requirements:

- **User Management**
 - Users can register, log in, and manage their profile
- **Role Management**
 - Support role-based access control (USER / ORGANIZER)
- **Event Management**
 - Organizers can:
 - Create events
 - View events
 - Update events
 - Delete events
- **Reservation Management**
 - Users can:
 - Browse available events

- Reserve seats
- **Input Validation**
 - Validate:
 - Name format
 - Email format
 - Password strength
 - Event details (title, date, location, capacity)
- **Duplicate Prevention**
 - Prevent multiple accounts with the same email
- **Authorization Rules**
 - Only organizers can manage events
- **Error Handling**
 - Display clear and user-friendly error messages

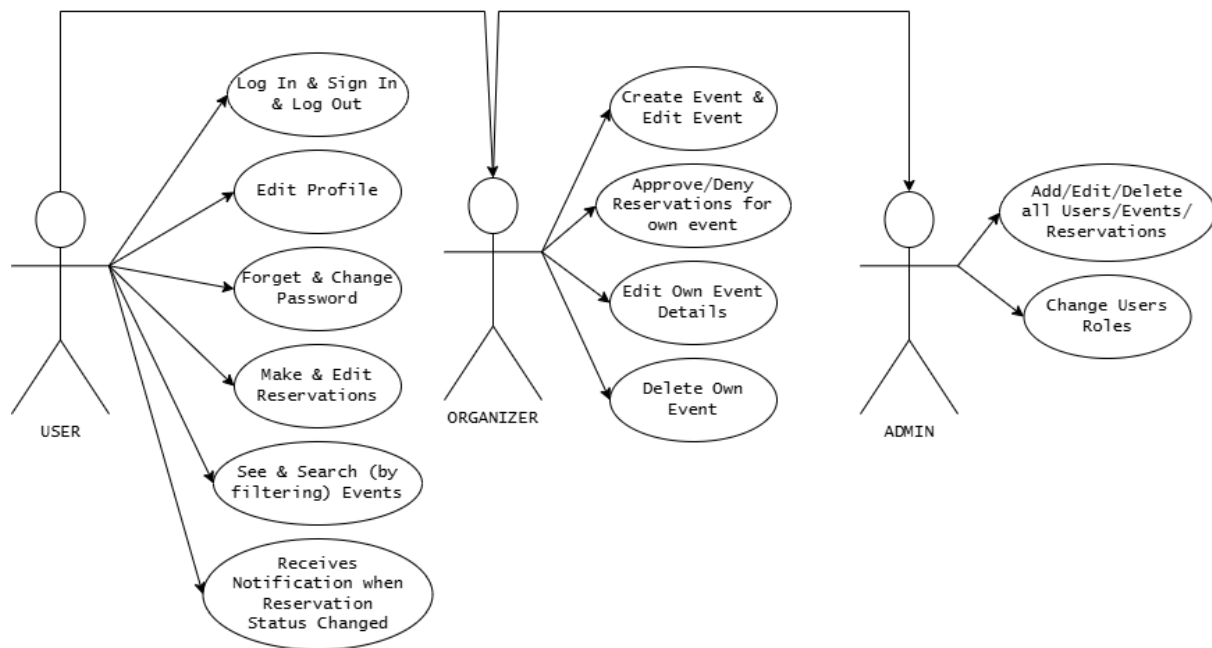
1.3 Non-functional Requirements

The application has the following non-functional requirements:

The application has the following non-functional requirements:

- **Layered Architecture**
 - Controller layer
 - Service layer
 - Repository layer
- **ORM Integration**
 - Use **Spring Data JPA**
- **Dependency Injection**
 - Use Spring DI container
- **Global Exception Handling**
 - Centralized error handling with meaningful HTTP responses
- **Scalability & Maintainability**
 - Modular and extensible design
- **Data Integrity**
 - Ensure consistency of reservations and event capacity
- **Security**
 - Authentication and role-based authorization

2. Use-Case Model



Basic User

- Creates account
- Edit profile details
- Recuperate password with email
- Browser or search by filtering events
- Makes, sees and edits own reservations

Organizer

- Inherits all basic user
- Views, creates, edits, deletes own events
- Views, edits, accepts or denies reservations from own events

Admin

- Can do everything a basic user and all organizers can do
- Change all users roles

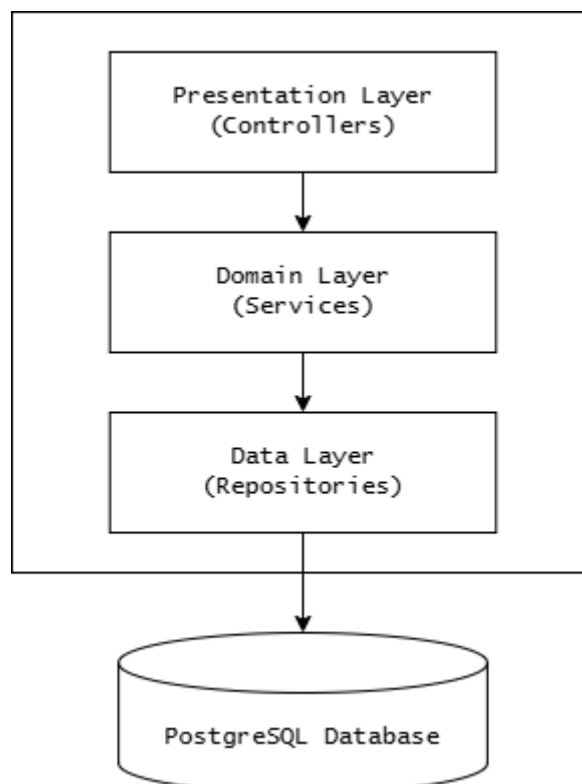
3. System Architecture Design

3.1 Architectural Pattern Description



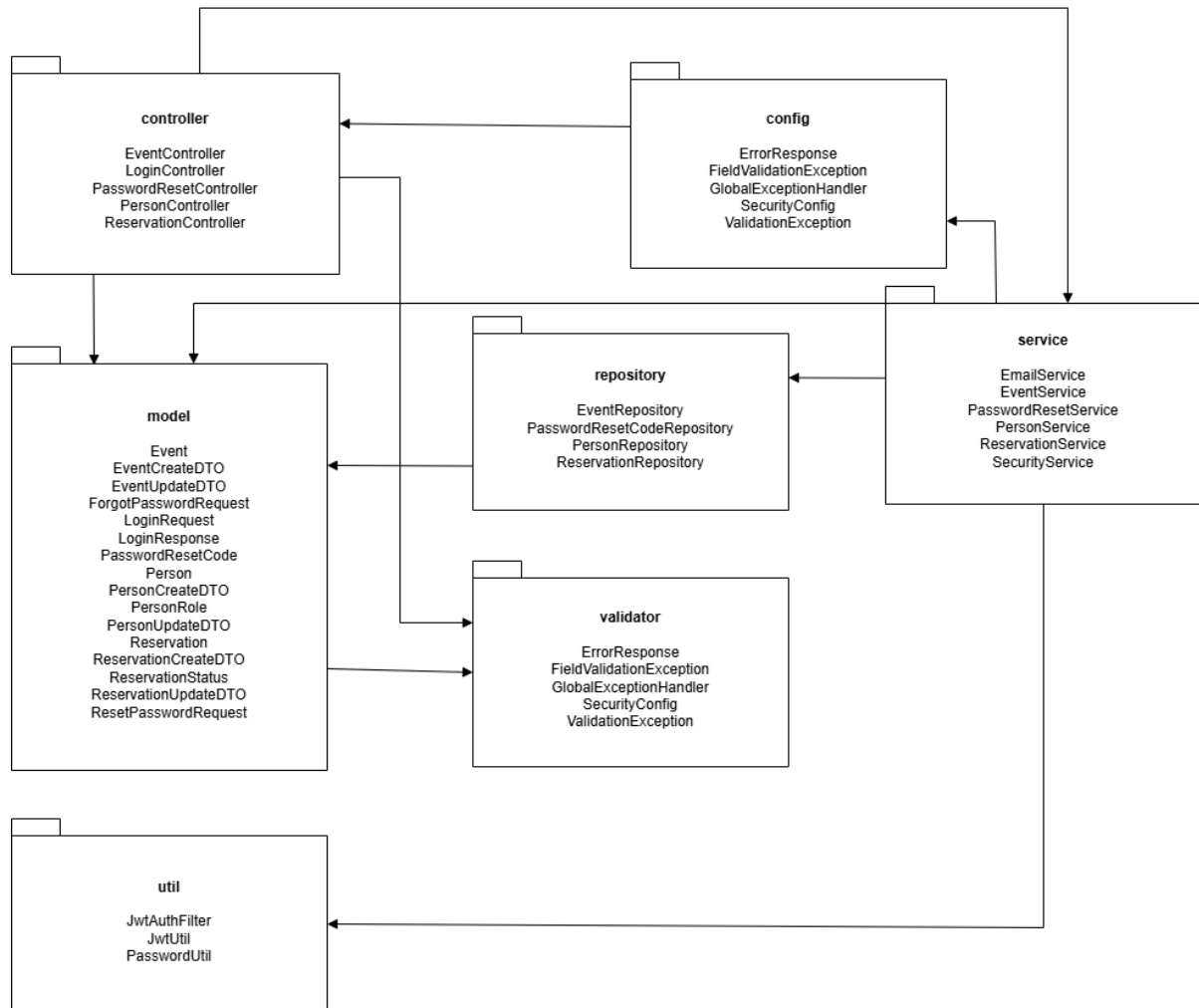
- **Presentation Tier (Client):** Implemented as a dynamic frontend using Angular. It handles user interactions, sends requests to the backend, and displays clear, user-friendly error messages when a request fails due to validation or server errors.
- **Logic Tier (Server):** Implemented as a robust backend with Spring Boot. This tier manages the core application logic, relying on a Dependency Injection (DI) container to manage component life cycles. It enforces strict input validation and features a centralized Global Exception Handler to capture business logic failures and map them to appropriate HTTP responses.
- **Data Tier (Database):** Uses a relational database integrated with Spring Data JPA (ORM) to handle all database interactions and entity mapping

3.2 Layered Architecture



- **Presentation Layer (Controllers):** The entry point for requests (e.g., PersonController).
- **Domain Layer (Services):** Where the business logic resides (e.g., ReservationService, EventService).
- **Data Layer (Repositories):** Handles the actual communication with the database using Spring Data JPA.

3.3 Package + Class Diagram



The application's backend architecture is organized into several key modules that enforce a clean separation of concerns. The modules are structured as follows:

- **Controller Layer:**

This package contains the REST Controllers (PersonController, CourseController, DepartmentController, AssignmentController, SubmissionController, LoginController, and PasswordResetController) which act as the entry point for requests from the presentation tier. These controllers map HTTP verbs to specific backend logic and delegate tasks to the appropriate services.

- **Service Layer:**

This module contains the business logic implementations (PersonService, EventService, ReservationService, SecurityService, PasswordResetService and EmailService). It acts as an intermediary, processing data, handling authentication checks, facilitating multi-step flows (like email-based password recovery), and managing transactions before data is sent to the repository.

- **Repository Layer:**

This module manages data access through Spring Data JPA interfaces such as PersonRepository, EventRepository, ReservationRepository and PasswordResetTokenRepository. These interact directly with the PostgreSQL database. (Note: Authentication and credential verification rely directly on the PersonRepository rather than a separate credentials repository)

- **Model Layer:**

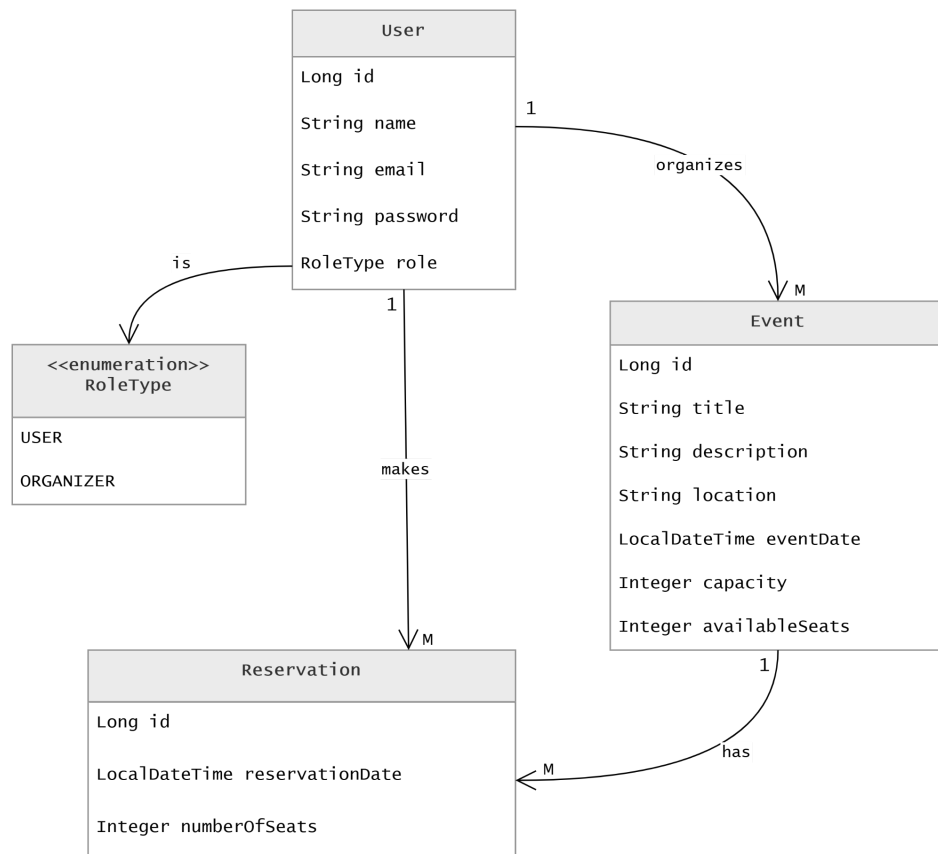
This central package defines the domain entities (Person, Course, Department, Assignment, Submission, PasswordResetToken) and the Data Transfer Objects (DTOs) used for communication. Specifically for authentication, it includes:

- LoginRequest: A DTO containing the user's credentials (email and password) sent from the client.
- LoginResponse: A DTO returned upon successful authentication, containing the user's assigned Role (admin, student, or professor), the generated JWT token, and the user's ID.

- **Config, Util & Validator:**

- **Config:** Contains global handlers like GlobalExceptionHandler to manage application-wide error responses. It also houses the security backbone, including SecurityConfig, JwtAuthFilter, JwtService, and PasswordEncoderConfig to manage stateless JWT authentication and CORS policies.
- **Util:** Contains helper classes like PasswordUtil to handle password hashing operations securely for the service layer.
- **Validator:** Contains custom validation logic, such as StrongPasswordValidator, to ensure data integrity and security before records are persisted to the database.

3.4 Class Relation



Person ↔ Event (One-to-Many)

A Person with the role ORGANIZER can create many Events, but each Event has exactly one organizer. This is a one-to-many relationship mapped via the `organizer_id` foreign key in the Event table. In JPA, the Event entity holds a `@ManyToOne` reference to Person.

Person ↔ Reservation (One-to-Many)

A single Person (with role USER) can have many Reservations — one per event they want to attend. Each Reservation belongs to exactly one Person. This is again a one-to-many, mapped by the `person_id` foreign key in the Reservation table.

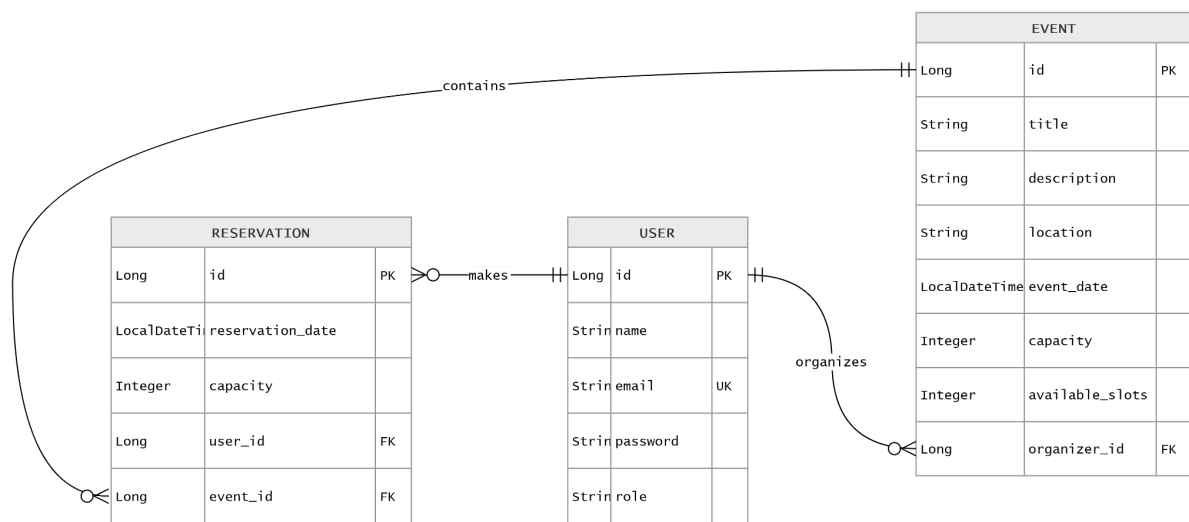
Event ↔ Reservation (One-to-Many)

A single Event can have many Reservations from different users. Each Reservation is tied to exactly one Event via the `event_id` foreign key. This is a one-to-many relationship.

Person ↔ Event (Many-to-Many through Reservation)

While there is no direct many-to-many mapping in the code, Person and Event are effectively connected in a many-to-many fashion through the Reservation join entity. A Person can reserve spots at many Events, and an Event can have many Persons reserving spots. The Reservation table acts as the associative/junction table, enriched with extra fields: `spots_reserved` and `status` (PENDING, ACCEPTED, DECLINED).

3.5 Data Model



User

- `id` - unique identifier
- `name` - full name of the user
- `email` - unique email address
- `password` - encrypted password
- `role` - role of the user (`USER` or `ORGANIZER`)

Event

- `id` - unique identifier
- `title` - name of the event
- `description` - event details
- `location` - place where the event will take place
- `eventDate` - scheduled date and time
- `capacity` - maximum number of seats
- `availableSeats` - remaining number of seats
- `organizer` - the user who created the event

Reservation

- **id** - unique identifier
- **reservationDate** - date and time when the reservation was made
- **numberOfSlots** - number of reserved seats
- **user** - the user who made the reservation
- **event** - the reserved event

4. Tests on Backend

Controller Integration Tests

PersonControllerIntegrationTests

- testGetPeople
- testAddPerson_ValidPayload
- testAddPerson_InvalidPayload
- testGetPersonById
- testGetPersonByEmail
- testPatchPerson_ValidPayload
- testDeletePerson
- testAddPerson_DuplicateEmail

EventControllerIntegrationTests

- testGetEvents
- testAddEvent_ValidPayload
- testAddEvent_InvalidPastDate
- testAddEvent_InvalidCapacity
- testGetEventById
- testPatchEvent_ValidPayload
- testDeleteEvent

ReservationControllerIntegrationTests

- testGetReservations
- testAddReservation_ValidPayload
- testAddReservation_InvalidSpotsReserved
- testGetReservationById
- testPatchReservation_ValidPayload
- testDeleteReservation

Service Unit Tests

PersonServiceTests

- testGetPeople
- testGetPersonById_ExistingId
- testGetPersonById_MissingId
- testGetPersonByEmail_ExistingEmail
- testGetPersonByEmail_MissingEmail
- testAddPerson_ValidPayload
- testAddPerson_DuplicateEmail
- testUpdatePerson_ExistingId
- testUpdatePerson_MissingId
- testPatchPerson_ValidPayload
- testPatchPerson_DuplicateEmail
- testDeletePerson
- testDeletePerson_MissingId

EventServiceTests

- testGetEvents
- testGetEventById_ExistingId
- testGetEventById_MissingId
- testAddEvent_ValidPayload
- testAddEvent_MissingOrganizer
- testUpdateEvent_ExistingId
- testUpdateEvent_MissingId
- testPatchEvent_ValidPayload
- testPatchEvent_WithOrganizerId
- testPatchEvent_MissingEventId
- testDeleteEvent
- testDeleteEvent_MissingId

ReservationServiceTests

- testGetReservations
- testGetReservationById_ExistingId
- testGetReservationById_MissingId
- testAddReservation_ValidPayload
- testAddReservation_MissingPerson
- testAddReservation_MissingEvent
- testUpdateReservation_ExistingId
- testUpdateReservation_MissingId

- testPatchReservation_ValidPayload
- testPatchReservation_MissingId
- testDeleteReservation
- testDeleteReservation_MissingId

SecurityServiceTests

- testLoginSuccess
- testLoginWrongPassword
- testLoginEmailNotFound

5. Changes for the project

For the final version of the project, I focused on improving the application beyond the basic backend and CRUD requirements. The main changes were made in two directions: improving the user experience of the frontend and adding a real-time notification mechanism using WebSocket.

The goal was to make HikeAlong feel more like a complete event reservation platform, not only a technical CRUD application. For this reason, I worked on making the interface easier to use, separating the flows based on user roles, and improving the reservation process with clearer feedback.

5.1 UI/UX Improvements

One of the most important changes was improving the frontend user experience. Since the project is demonstrated mainly through the interface, I wanted the application to be easy to understand for each type of user.

The application now has a role-based interface. After login, the user is redirected based on their role, and the toolbar displays only the actions that make sense for that role.

For a basic **USER**, the interface focuses on:

- browsing events;
- searching and filtering events;
- making reservations;
- seeing their own reservations;
- editing their profile;
- receiving reservation status updates.

For an **ORGANIZER**, the interface focuses on:

- creating events;
- viewing their own events;
- editing events;
- deleting events;
- viewing reservations made for their events;
- accepting or rejecting reservation requests.

For an **ADMIN**, the interface allows access to a broader set of management actions, such as managing users, events, reservations, and changing user roles.

This role-based UI was chosen because it makes the application clearer and avoids showing unnecessary actions to users. An alternative approach would have been to show the same interface for all roles and only block actions in the backend. However, that would make the frontend more confusing. The chosen solution improves usability while the backend still keeps the important validation and authorization logic.

Another important UI/UX improvement was the event browsing page. The event list is one of the main screens of the application, so I improved it to make events easier to find and understand. Users can browse available events and use search, filtering, and sorting features.

The Event entity was chosen for search, filtering, and sorting because it is the most important entity from the user's point of view. A user usually starts by looking for an event, so it makes sense to help them find events faster.

The event browsing page supports or is designed to support:

- searching events by title;
- filtering events by location;
- filtering upcoming events;
- sorting events by date;
- viewing event details in a clearer format;
- making reservations directly from the event flow.

I chose this approach because it improves the user experience without changing the core backend model. For a small demo dataset, frontend filtering is simple and fast. For a larger real application, the same filtering logic could later be moved to the backend using repository queries.

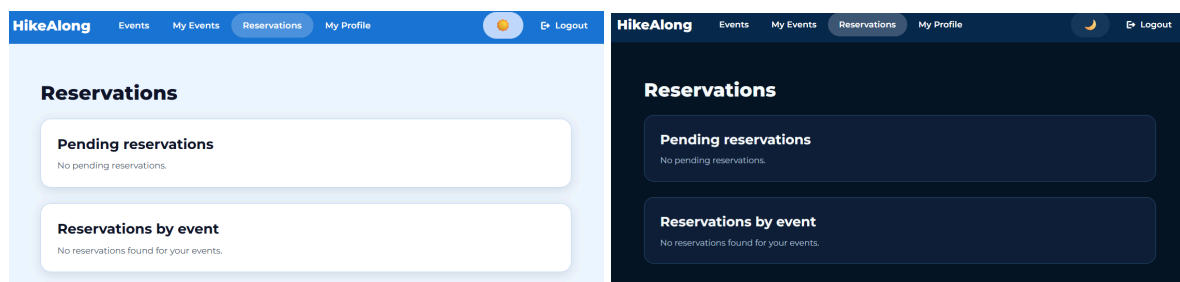
I also added clearer visual feedback using notifications. Instead of leaving the user unsure whether an action succeeded or failed, the frontend displays messages for important actions. For example, the user receives feedback after login, after creating a reservation, after an event is created or updated, and after a reservation status is changed.

This improves the user experience because the user does not have to inspect the database or refresh the page manually to understand what happened. The interface communicates the result directly.

Forms were also improved with validation feedback. The application validates data such as name, email, password, event title, event date, event capacity, and reserved spots. When something is invalid, the system shows a clear error message instead of failing silently.

The main **UI/UX design** decisions were:

- separate interfaces based on role;
- hide actions that are not relevant for the current user;
- make the event feed the main page for users;
- make event and reservation management the main pages for organizers;
- provide visible feedback after every important action;
- keep the interface simple enough for a live demonstration;
- use search, filtering, and sorting to make the application feel more complete.



5.2 WebSocket Notification Feature

Another important change was adding a WebSocket notification feature for reservation updates.

The purpose of this feature is to improve the communication between organizers and users. In the normal reservation flow, a user creates a reservation and the organizer later accepts or rejects it. Without real-time notifications, the user would need to manually refresh the page or check the reservation list repeatedly to see whether the status changed.

With WebSocket notifications, the backend can send a message when the reservation status changes. The frontend can then display this message immediately using the notification/toast system.

The reservation **notification flow** is:

1. A USER creates/edits a reservation for an event.
2. The reservation is saved with the status PENDING.
3. The ORGANIZER views the reservations for their events.
4. The ORGANIZER accepts or rejects the reservation.
5. The backend updates the reservation status to ACCEPTED or DECLINED.
6. The backend sends a WebSocket notification.
7. The frontend receives the notification.
8. The notification is displayed to the user through the interface.

Example messages are:

- "Your reservation was accepted."
- "Your reservation was rejected."
- "A reservation status has been updated."

```
=====
RESERVATION NOTIFICATION SENT
Reservation id: 8736b518-941e-4ebb-b217-711bd797b123
User email: user2@gmail.com
Event: Via Ferrata Adventure in Cheile Turzii
Status: ACCEPTED
Destination: /topic/user/user2@gmail.com/notifications
Message: Your reservation for Via Ferrata Adventure in Cheile Turzii was accepted!
=====
```

A design decision I made was to keep WebSocket as an additional notification layer, not as the main way of saving data. The actual reservation update is still done through the REST API. WebSocket is used only to inform the frontend about the change.

This is important because the application should still work even if the WebSocket connection is not active. If WebSocket fails, the reservation status is still saved in the database, and the user can still see the updated status by refreshing the reservation page. This makes the design more reliable.

I considered two possible approaches for reservation updates:

- polling;
- WebSocket notifications.

Polling would mean that the frontend sends repeated requests to the backend every few seconds to check if the reservation status changed. This approach is simpler, but it creates unnecessary requests and is not truly real-time.

WebSocket is a better choice for this project because reservation status updates are event-based. A notification is needed only when something actually changes. This makes WebSocket more efficient and more suitable for real-time feedback.

The WebSocket feature also improves the project from a demo perspective. It shows that the system is not limited to simple REST CRUD operations, but also includes a real-time communication mechanism.

5.3 Reservation Flow Improvements

The reservation flow was also improved to make it clearer and more realistic.

The Reservation entity connects a Person to an Event and stores:

- the person who made the reservation;
- the event that was reserved;
- the number of reserved spots;
- the reservation status.

The status can be:

- PENDING;
- ACCEPTED;
- DECLINED.

This status-based design was chosen because it gives organizers control over reservations. Instead of every reservation being automatically accepted, the organizer can decide whether to accept or reject it. This makes the application more realistic for events where the organizer may want to manually approve participants.

The reservation flow also gives the WebSocket notification feature a clear purpose. The notification is triggered when the status changes, which is an important moment for the user.

5.4 Final Summary of Project Changes

In conclusion, the main project-specific changes were focused on making the application more complete and easier to use.

The most important improvements were:

- role-based frontend navigation;
- clearer separation between USER, ORGANIZER, and ADMIN flows;

- improved event browsing page;
- search, filtering, and sorting for events;
- improved reservation flow with PENDING, ACCEPTED, and DECLINED statuses;
- organizer reservation management;
- WebSocket notifications for reservation status changes;
- toast notifications for user feedback;
- clearer validation messages;
- better UI/UX for the live demonstration.

These changes make HikeAlong more than a simple event CRUD application. The final version behaves more like a real event reservation platform where users can easily find events, make reservations, and receive updates, while organizers can manage both events and reservation requests.

6. Vision and Supplementary Specification

The vision of HikeAlong is to provide a complete full-stack platform for managing hiking events and reservations in a simple, structured, and user-friendly way. The application is designed for people who want to discover hiking events and reserve seats, but also for organizers who need to create events, manage them, and handle reservation requests.

The main idea of the project is to replace informal or manual event management methods, such as messages, spreadsheets, or separate lists, with one centralized web application. In HikeAlong, all important information about users, events, and reservations is stored and managed in the same system.

The application focuses on three main types of users: basic users, organizers, and admins.

A **basic user** can create an account, log in, edit their profile, recover their password by email, browse events, search and filter events, make reservations, view their own reservations, and receive updates about reservation status changes.

An **organizer** inherits the basic user functionality, but also has access to event management features. An organizer can create events, view their own events, edit event details, delete events, view reservations made for their events, and accept or reject reservation requests.

An **admin** has the highest level of access. The admin can manage users, events, reservations, and user roles. This role is useful for keeping control over the entire application and for demonstrating full system management.

The project is built as a full-stack application. The backend is implemented using Spring Boot and is responsible for authentication, validation, business logic, database communication, and error handling. The frontend is implemented using Angular and is responsible for the user interface, role-based navigation, forms, event browsing, reservation actions, and notifications.

The main entities of the system are Person, Event, and Reservation.

The Person entity represents a user account and stores information such as name, email, password, age, and role. The role determines what the user is allowed to do inside the application.

The Event entity represents a hiking event. It stores information such as title, description, location, date, maximum number of participants, and organizer. Events are the central part of the application because users interact mainly with the event feed.

The Reservation entity connects a user to an event. It stores the number of reserved spots and the reservation status. A reservation can have the status PENDING, ACCEPTED, or DECLINED. This makes the reservation flow more realistic because organizers can manually accept or reject requests.

One important goal of the project is to offer a clear user experience. The frontend interface changes depending on the user role. A basic user sees event browsing and reservation actions, while an organizer sees event management and reservation approval actions. This makes the application easier to use and avoids showing unnecessary buttons or pages.

Another important part of the vision is improving event discovery. Users should not only see a static list of events, but should be able to search, filter, and sort them. The Event entity was chosen for search and filtering because it is the most important entity from the user's point of view. Users need to quickly find events based on title, location, date, or availability.

The project also includes real-time communication through WebSocket notifications. When a user makes a reservation, the reservation is initially pending. After the organizer accepts or rejects it, the backend sends a notification, and the frontend displays it to the user. This improves the experience because the user does not need to constantly refresh the page to check if the reservation status changed.

The WebSocket functionality is designed as an additional notification layer. The main reservation update is still handled through the REST API, while WebSocket is used only to send real-time feedback. This design makes the system more reliable because the reservation status is still saved even if the notification connection is temporarily unavailable.

The application also focuses on validation and clear error handling. The system validates user data, email format, password strength, event information, event date, event capacity, and reservation details. When something is wrong, the user should receive a clear error message instead of an unclear backend failure.

From a technical perspective, the vision of the project is to keep the system modular and maintainable. The backend follows a layered architecture with controllers, services, repositories, models, DTOs, validators, and exception handling. This separation makes the code easier to understand, test, and extend.

The frontend is also organized around clear responsibilities: login, event feed, profile, organizer event management, reservation management, and notifications. The goal is to make the application easy to demonstrate and easy to use.

Overall, HikeAlong is designed as a realistic event reservation platform. It combines backend functionality, frontend usability, role-based access, validation, search and filtering, reservation management, WebSocket notifications, and clear UI feedback. The final goal is to provide a complete application where users can easily discover hiking events and make reservations, while organizers can efficiently manage their events and reservation requests.